
Machine Learning for Quantum Machine Calibration Forecasting

Garrigues-Journaud Gwenn

Supervised by Dr. Zakaria Abdelmoiz DAHI
Researcher, INRIA, BONUS



Master's Research Project

Data Science / First year

University of Lille, Centrale Lille, IMT Nord Europe

Contents

1	Introduction	3
1.1	Context	3
1.2	Objectives	3
2	Fundamental Notions	4
2.1	Basics of Quantum Computing	4
2.2	Time Series Forecasting Machine Learning Method	5
3	Extraction-Load-Transform (ELT)	5
3.1	Extraction	6
3.1.1	Quantum Hardware Suppliers	6
3.1.2	Scraping from IBM	7
3.2	Loading	8
3.2.1	Calibration Metrics	8
3.2.2	Load and Save	8
3.3	Transformation	9
3.3.1	Analysing the Frequency	9
3.3.2	Correlation and Feature Analysis	11
4	Modeling and Learning	12
4.1	Feature Engineering and Training Dataset	12
4.2	Models	13
4.2.1	Recurrent Neural Networks (RNN)	13
4.2.2	Statistical Models	13
4.2.3	Neural Prophet	15
4.3	Experimental	15
4.3.1	Hardware and Benchmark Settings	15
4.3.2	Training	16
4.3.3	Benchmarking	16
4.4	Result	17
5	Conclusion and Perspectives	20
6	Appendix	24

List of Figures

1	Bloch Sphere representation	4
2	Visual representation of the ELT pipeline.	6
3	Forecasting of the T1 qubit median using a GRU model on hourly calibration data (ibm_fez, time lags: 2 steps)	18
4	Forecasting of the T1 qubit median using Neural Prophet model on hourly calibration data (ibm_fez)	18
5	Forecasting of the T1 qubit median using a GRU model on daily calibration data (ibm_fez, time lags: 5 steps, using all the features)	19
6	Prediction of T1 with SARIMAX using exogenous variables (Fez - Daily)	19
7	Correlation matrix – Fez on data scraped for 42 days, after feature engineering.	24
8	Correlation matrix – Torino on data scraped for 42 days, after feature engineering.	24
9	Correlation matrix – Marrakesh on data scraped for 42 days, after feature engineering.	24
10	Visual analysis of the calibration stability of the three backends with a week-by-week comparison of cumulative calibration changes with the median of the stability time for each of them and our theoretical optimal scraping frequency of one hour, 42-day dataset scraped with a frequency of 1 min.	25

List of Tables

1	Overview of cloud-based quantum hardware providers	6
2	Set of calibration metrics provided by IBM API ¹	8
3	Calibration stability over 42 days per backend computed with (2)	10
4	ADF Stationarity Test Results by Backend and Feature	14
5	Hyperparameter settings per model and dataset granularity	16
6	Comparison of models across calibration metrics provided by the benchmark, averaged over all backends (hours).	17
7	Comparison of models across calibration metrics, averaged over all backends (daily dataset).	18
8	Comparison of calibration metrics for NeuralProphet, averaged over all the backends (days)	20
9	Descriptive statistics of qubit metrics for ibm_fez	26
10	Descriptive statistics of gate metrics for ibm_fez (<i>error values expressed in %</i>)	26
11	Descriptive statistics of qubit metrics for ibm_torino	26
12	Descriptive statistics of gate metrics for ibm_torino (<i>error values expressed in %</i>)	26
13	Descriptive statistics of qubit metrics for ibm_marrakesh	27
14	Descriptive statistics of gate metrics for ibm_marrakesh (<i>error values expressed in %</i>)	27

1 Introduction

In this section, we will introduce the current context of our project and the challenges it faces. We will discuss the fundamentals of quantum computing, its advantages, and its limitations. We will then outline the current state of the art related to our research problem before presenting our approach to addressing it.

1.1 Context

Quantum machines are similar to classic computer, using instead of classic bits, qubits (quantum bits). A bit can only be in state 0 or 1, while the state of a qubit is a linear combination of the two basic states, allowing a significant acceleration of calculations in specific cases. This acceleration that quantum computers promise would make it possible to advance many areas requiring significant amounts of calculations, such as optimization, cryptography and simulation.

However, quantum computers nowadays are imperfect, qubits and gates are very sensitive to noise related to their environment. They are in what we call, their Noisy Intermediate-Scale Quantum (NISQ) Era. This noise strongly affects the computation of quantum computers and leads to high error rates, which is why regular re-calibrations of the physical state of the machine are necessary. One of the most promising and scalable computation technology is the superconducting qubits. However, this computation hardware is also quite sensitive to noise and therefore to errors.

Machines being difficult to access and expensive, most people performing quantum calculations go through cloud service providers in order to send their algorithms to be executed remotely on their physical machines. Access to these machines being paid, the instability of these machines causes errors and therefore leads to a loss of time and money.

1.2 Objectives

One of the ways to mitigate this instability related to noise affecting our machines would be to predict the temporal evolution of this instability affecting quantum computers. Some suppliers of quantum machines share the evolution of calibration metrics, these metrics assess the state and sanity of the machines. The use of modern machine learning methods could be a solution to predict the optimal time when to run a computation by predicting the calibration state of a machine over time, maximizing the accuracy of computations and allowing significant savings in money and time.

The work of Dahi et al. [3] is directly precursor to this project. Their contribution explore the idea of mitigating quantum errors without modifying the computation or the machine itself, but instead by predicting the moments when the machine is in its most stable state. Their pipeline relies on the temporal forecasting of qubits calibration metrics (T1, T2, readout errors, etc.) extracted from three real IBM quantum machines over 3.5 months, using LSTM and Neural Prophet models. Among their identified future directions, they explicitly mention extending the study to wider range of models, longer data collection periods, and other quantum systems. The present work addresses these open challenges directly.

Complementary to the work of Dahi et al., similar studies have focused on predicting the evolution of calibration metrics for two-qubit gates using various machine learning and time-series forecasting techniques, using IBM calibration data for a longer period (9 months) [10]. Others have also explored mitigating errors in circuit outputs through predictive modeling approaches [8].

Our approach consists of setting up a complete pipeline with an Extraction-Load-Transform (ELT) phase, including the extraction of calibration metrics over several weeks, data cleaning and persistent storage. Followed by an Exploratory Data Analysis (EDA) phase, which allows us to extract insights from the data and better understand its behavior. This is followed by a modeling and learning phase, including a feature engineering part based on the previous observations, benchmarking different models for time series forecasting (neural, non-neural, statistical, and sequential) at different levels of granularity, and finally a performance evaluation.

2 Fundamental Notions

This section will provide additional context and fundamental notions of quantum computing and machine learning for time series forecasting.

2.1 Basics of Quantum Computing

The fundamental unit of a quantum computer, *the qubit*, can be represented as a combination of two basis states, $|0\rangle$ and $|1\rangle$. We represent its state as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ with $\alpha, \beta \in \mathbb{C}$ the amplitude of the probability of being in the corresponding state. $|0\rangle, |1\rangle$ can be geometrically represented on the so-called *Bloch Sphere* where each point on its surface corresponds to a valid qubit state (see Figure 1). Represented as a point on a sphere, we can use transformations on these qubits by using different methods according to the paradigm we use. The *gate-based* paradigm uses gates to apply transformations on the qubit such as rotations or measurements. In a gate-based paradigm, algorithms are represented as sequential combinations of gates called *circuits*. The X-gate for example takes a qubit and transforms $|0\rangle$ into $|1\rangle$ and $|1\rangle$ into $|0\rangle$, this corresponding to a rotation of the qubit around the X-axis.

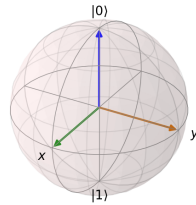


Figure 1: Bloch Sphere representation

Qubits can only maintain the excited state $|1\rangle$ for a limited period of time, characterized by T1, the *Relaxation Time*. On the other hand, qubits are affected by environmental noise and lose precision as time passes, this instability is characterized by T2, the *Dephasing Time*. Since the gates that measure the qubit are not perfect and are affected by environmental noise, the qubit can be wrongly measured. This error is called the *Readout Error* or *Measurement Error*. Each operation on a qubit such as a transformation or a

measurement operated by a gate takes time and is not perfect, hence each gate has a *Duration* and an *Error Rate*. Often, the longer you measure the gates, the more precise you will be. However, since qubit stability is limited over time, a trade-off must be found between precision and readout length [6]. There are different technologies to represent *qubits*. One of the most widely used is the **superconducting** qubit, which exploits the quantum properties of electrical circuits in an environment close to 0 Kelvin (*absolute zero*). Other technologies exist, such as **trapped-ion** qubits, which are more stable and use ions to encode quantum states but are slower in terms of computation, or photonic qubits, using photons. Superconducting qubits are admittedly more unstable but faster in computations, which is why predicting this instability is a key challenge [5].

2.2 Time Series Forecasting Machine Learning Method

Time series are sequence of values represented according to a temporal evolution and can be modeled as a signal. A time series can be expressed on different bases (hourly, daily, monthly). Time series forecasting use the past behavior of data to forecast the current value. Forecasting values found applications across many domains, including economics with prices prediction, healthcare with estimation of behavior of disease or environmental purpose. Machine learning methods for time series forecasting can be divided in two categories. The first consists of *Recurrent Neural Network* models such as *LSTM* and *GRU*, referred to *non-parametric models* [1, 7]. Using gates mechanisms and memory systems, these models exploit temporal information to capture non-linear patterns with the temporal structure inherent in the data.

The second group, known as *parametric models*, relies on statistical properties to understand the distribution of the data and build models based on signal decomposition. These models express current values as a linear combination of past values and errors to construct predictive frameworks. Prior knowledge is required to set the hyperparameters governing both signal decomposition needed for the prediction of future values. This necessitates the use of various statistical methods to study these properties such as seasonality, stationarity, and the influence of past values on present ones. Statistical models such as *SARIMAX* have demonstrated strong utility in the field of time series forecasting [4]. As both families present distinct advantages, *Neural Prophet* represents a fusion of the two categories into a single model that combines the strengths of each.

3 Extraction-Load-Transform (ELT)

In this section, we will explore the dataset construction pipeline divided in three step. We first describe the *Extraction* process of the raw calibration data, following by the *Loading* part about the persistence and storage of the data, and conclude with the *Transformation* part, guided by our Exploratory Data Analysis (EDA), covering data analysis and feature engineering.

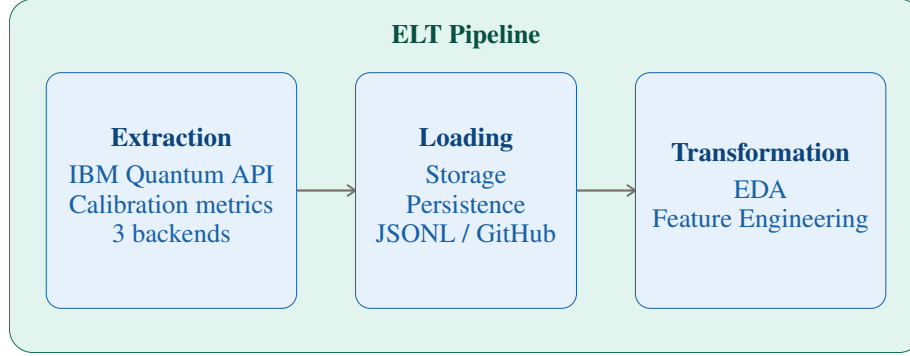


Figure 2: Visual representation of the ELT pipeline.

3.1 Extraction

This part explain the methodology used to extract the calibration data from the IBM Quantum backends.

3.1.1 Quantum Hardware Suppliers

The first part of this project was to find suppliers for quantum hardware under certain conditions. The backends used must be real machines, since simulators do not faithfully reproduce the behavior of actual quantum computers. The backends must use a *gate-based* quantum circuit paradigm. The selected backends must use *superconducting* qubits and the providers must publicly share their calibration data.

We chose to select only superconducting qubits because their instability represents a more critical challenge for quantum computation. Furthermore, it is not possible to compare the two technologies, as they do not use the same values, metrics, and measurement methods. After these conditions were defined, among all the quantum hardware providers we found, only a few remain worth considering. Table 1 presents the summary of the identified providers, the technology they use and the type of access they provide.

Provider	Technology	Access	Selected
IBM Quantum	Superconducting	Public API / High level SDK	✓
IQM	Superconducting	Public API / High level SDK	✓
Quantum Inspire	Superconducting	Public API	✓
Microsoft Azure	Superconducting	Private	×
IonQ	Trapped ions	Public API	×
Quantinuum	Trapped ions	Public API	×
Oxford Quantum Circuits	Superconducting	Special access only	×

Table 1: Overview of cloud-based quantum hardware providers

After contacting engineers from **Oxford Quantum Circuits** in order to obtain restricted access solely for gathering calibration data, they indicated that such access was not possible within the context of this project. Finally, we keep at the end **IBM**, **IQM** and **Quantum Inspire**, being the only three satisfying our

condition and sharing their calibration data for free.

3.1.2 Scraping from IBM

This section describes the scraping module of the extraction part. Using the python package *Qiskit*, we scraped the calibration metrics provided by the *IBM Quantum Platform* to create the dataset. IBM provide a high-level SDK with a python package based on Qiskit to extract these calibration metrics from the API, hence the scraping was done in Python. Extracting these data with Qiskit requires to use an API key provided by IBM in order to connect to their physical backend. IBM API provides us with several calibration metrics and additional information:

- The backend name : (Fez, Torino, Marrakesh).
- The number of qubits : variable for each backend (*156 for Fez and Marrakesh, 133 for Torino*).
- Qubit and gates calibration metrics.

To this we add another features called **calibration_time** which give us the temporal information of the moment the calibration were extracted and would be useful to easily find a calibration state.

IQM aswell as IBM has their own Qiskit version. However, the Qiskit package of IQM is not as well made as IBM. Some information such as the frequency and the anharmonicity are not provided and their package didn't provide the same level of SDK allowing us to extract raw data with no real explanation of each value's meaning. *Quantum Inspire* did not provide any high-level SDK. The only way to extract these datas was to explore the API and to web scrape their website. After contacting them, they confirmed not providing this information.

One of the major problems for *IQM* and *QuantumInspire* is that they provide access to the API mainly to allow quantum circuits to run, hence the calibration shared from the API are poorly documented. However, even if *IQM*, *IBM* and *Quantum Inspire* respect our previously defined conditions, two majors inconsistencies can happen while comparing their calibration data :

- **Gate heterogeneity (lack of universality):** Each providers didn't provide computer based on the same gate sets. (Ex: IQM use a gates sets with the 2-qubits gates PRX and CZ only while IBM use a different set.
- **Metric Ambiguity (lack of measurement standardization):** Even if providers use the same gate set's or only a few different, they can use different measure protocol to compute the same calibration metrics. For instance, IBM and IQM provide calibration information related to the **T2** metrics, which can be measured either via *Ramsey* or *Echo* methods, giving different values. Most of the time, provider didn't provide which type of measure they use. In this case, it's not really possible to compare different backend from different providers, making it impossible to adjust or normalize the data accordingly.

Since these inconsistencies represent a limitation for cross-providers comparison, we conclude that the best choice was to focus on *IBM* at first and then to generalize to other provider's backend.

3.2 Loading

This part describes the calibration metrics we extracted and how we ensure their persistence.

3.2.1 Calibration Metrics

The first step after finding the providers was to identify which information would be relevant and useful for our project. IBM backends expose a set of calibration metrics at both the qubit and gate levels (see Table 2).

Metric	Level	Description
T1 (Relaxation time)	Qubit	Time a qubit remains in its excited state $ 1\rangle$ before decaying to its ground state $ 0\rangle$
T2 (Dephasing time)	Qubit	Time a qubit can maintain a superposition state. Always satisfies $T_2 \leq 2 \times T_1$. [6]
Frequency	Qubit	Resonant frequency of a qubit, represent the energy difference between $ 0\rangle$ and $ 1\rangle$ (~ 5 GHz).
Anharmonicity	Qubit	Frequency difference between two states
Readout assignment error	Qubit	Average probability of incorrectly measuring a qubit state, computed as the mean of $P(0 1)$ and $P(1 0)$
$P(0 1)$	Qubit	Probability of measuring $ 0\rangle$ when the qubit is supposed to be in $ 1\rangle$
$P(1 0)$	Qubit	Probability of measuring $ 1\rangle$ when the qubit is supposed to be in $ 0\rangle$
Readout length	Qubit	Duration of the readout operation for a given qubit
ID error	Gate	Error rate of the identity gate, proportionally equal to the $\sqrt{X}$ and X gates
Single-qubit gate error	Gate	Error rate of single-qubit gate operations (e.g., $\sqrt{X}$, X gates)
Single-qubit gate length	Gate	Duration of a single-qubit gate operation
RZ error	Gate	Error of the virtual Z-axis rotation gate. (Reported as 0 when performed in software)
RZZ error	Gate	Error of the RZZ gates (<i>2-qubit gate</i>)
RZZ gate length	Gate	Duration of the two-qubit RZZ gate operation
CZ error	Gate	Error of the CZ gates (<i>2-qubit gate</i>)
CZ gate length	Gate	Duration of the two-qubit CZ gate operation
reset_error	Gate	Error of the reset gate (<i>resets a qubit to its ground state $0\rangle$</i>)

Table 2: Set of calibration metrics provided by IBM API²

At first, we decided to extract all of these metrics to ensure no valuable information would be missed.

3.2.2 Load and Save

Since the first objective was to create a dataset to forecast the evolution of the calibration metrics over time, our first goal was to automate the periodic extraction of data.

Frequency: With no prior knowledge of the calibration update frequency, we chose to scrap as frequently as possible to ensure not missing any variation, starting with a scraping frequency of *one minute*. However, we later found that scraping every minute was excessive since the calibration don't change that often. A scraping frequency of *one hour* was more adapted, as seen by our exploratory data analysis. The first approach was to scrape every minute and to locally run a Python script with a *Linux crontab*, saving the files on hardware. However, this quickly become limited. Then, we finally migrated to *Github Actions*, allowing to run the whole pipeline remotely, with the data stored directly inside *Github* using *Git Large File Storage (LFS)* to deal with the limited storage of Github of (1GB per repository).

²Metrics and their definitions sourced from IBM Quantum Platform documentation: <https://quantum.cloud.ibm.com/docs/en/guides/gpu-information>

Storage: For IBM we collect calibration data from 3 backends (Fez, Torino and Marrakesh), extracting for each of them, all the calibration metrics for all the qubits at each time. Since each backend contains between 133 and 156 qubits, each of them associated to multiples calibration metrics. In addition, each gates provides two metrics (*duration* and *error*) with a different value for each qubit. Two-qubits gates such as RZZ and CZ have values defined for pair of qubits

At each time T we scraped all the calibration metrics of all the qubits from the 3 backends which give us :

$$3 \text{ backends} \times 130 \sim 154 \text{ qubits} \times 26 \text{ features} \approx \mathbf{12012} \text{ values / hour} \quad (1)$$

To build our dataset incrementally, we choose to concatenate calibration data over periods of 2 weeks before merging them into the complete dataset files. For clarity and modularity, we choose to create 3 different datasets a *Qubits Dataset* containing the qubit related calibration metrics, a *Gates Dataset* with the gates related calibration metrics and a *Complete Dataset* providing both informations. Allowing us to load the file adapted to our needs, saving memory and providing modularity in the case we don't want to work with the whole dataset.

At first using CSV format, it become time-expensive to add new data in it as the dataset grows since we need to read it to add data. To deal with this, we changed to *JSONL*, a format allowing us to add value without loading the whole file. The switch to *CSV* $\rightarrow$ *JSON* $\rightarrow$ *JSONL* required the implementation of Python algorithm to transform the first weeks of data in the old format to the new format. To ensure, each line in the dataset are uniquely identified, we create a unique identifier of the form : `backend_name_scraping_time`.

3.3 Transformation

This section represents the last step of our ELT pipeline, mainly driven by the *Exploratory Data Analysis* (EDA). We will explore and analyse the data collected over 42 days of scraping. In the first part we will adapt the scraping frequency by studying the rate of change in our data. Then, we will look for patterns and correlation between the features to retain the most relevant and guide our modeling choices.

3.3.1 Analysing the Frequency

Using our complete dataset scraped from 29-11-2025 to 10-01-2026 (42 days), we split it in three dataframes for each backend sort by `calibration_time`. To analyse the rate of change of the calibration data, we computed the median of each calibration metrics, strongly reducing the number of columns of the dataframes. In order to find the optimal scrap frequency, we implemented a calibration stability analysis. The goal of this algorithm was to compare under a threshold, if a calibration at a timestamp t is changed at the next timestamp $t + 1$.

This method iterate over each `calibration_time` and compute a MSE (Mean-Square Error) between the features of t and $t + 1$, such that :

$$\text{MSE}(t, t + 1) = \frac{1}{n} \sum_{i=1}^n \left(\hat{x}_i^{(t)} - \hat{x}_i^{(t+1)} \right)^2 \quad (2)$$

with n the number of calibration metrics we use to compute this MSE, $\hat{x}_i^{(t)}$ representing the normalized calibration metrics at index i (normalized with min-max scaling method) extracted at the time t . We choose to normalize the data to ensure each feature contributes equally, regardless of their original scale. We consider

a calibration change if the MSE computed between 2 timestamps, exceed a threshold ϵ . After, empirically testing several values we set it to 0.001, providing the best compromise between not missing real change and not been affected by small perturbation not related to real change in the measure.

$$C = \{t \mid \text{MSE}(t, t+1) > \epsilon\} \quad (3)$$

with C the set of timestamp where a change in the calibration from their preceding values is detected. After saving all of these changes, we compute the average time between 2 variations and count the number of different calibration.

Backend	Changes detected	Mean stability	Median stability
ibm_fez	284	3h 30min	2h 47min
ibm_torino	332	3h 00min	2h 27min
ibm_marrakesh	374	2h 41min	2h 12min

Table 3: Calibration stability over 42 days per backend computed with (2)

The Table 3 represents the results of this stability analysis. It shows us that a scraping frequency of 1 minutes was excessive. To validate the consistency of our observations across the weeks, we analyze this stability week-by-week. We computed for each week the cumulative number of variations in the calibration for a 168h period (a week) and compared it against a theoretical value (See in the Appendix, Figure 10). These results show that scraping every minute was unnecessarily frequent given the actual calibration update rate. A scraping frequency of *one hour* was a reasonable compromise, capturing most of the calibrations changes while keeping a manageable volume of data.

Finally, these observations were confirmed by the IBM Quantum documentation <https://quantum.cloud.ibm.com/docs/en/guides/calibration-jobs> which runs two different type of calibration algorithms:

- **Hourly calibrations:** Checking the stability almost every hours, and taking 2-3min long (mainly focused on the qubits)
- **Daily calibration:** Full recalibration of the machine parameters, qubits benchmarking and gates calibrating/benchmarking. Since qubits properties are instable, IBM has to re-calibrate them once a day to minimize the error.

Furthermore this confirm our theoretical frequency of one hour and explains the difference observed in our stability analysis, while qubit-level metrics such as T1, T2, and readout errors are subject to hourly updates, gate-level metrics are re-calibrated on a daily basis. The fact that median and the mean exceed 1h is related to the way benchmark algorithms are interleaved with user-submitted jobs in the execution queue as explained in the documentation. In this case, the calibration update can be delayed and shared with the users after the scheduled time.

Since we reduced the scraping frequency, we still had access to previously collected data at a one-minute resolution. To align these data with the new sampling frequency, we aggregated them by computing the median of each feature over one-hour intervals. As the metrics collected at a one-minute frequency remained

stable over several hours, this approach allowed us to re-use the previously collected data without significant loss of information.

3.3.2 Correlation and Feature Analysis

After this, we analyze the behaviors of each features from all the backends to seek for null values, outliers, and/or patterns (see Table 9, Table 10, Table 11, Table 12, Table 13 and Table 14 in Appendix). Gate duration metrics (`id_duration`, `rx_duration`, `sx_duration`, `cz_duration`, `rzz_duration`, `measure_duration`) were excluded from the descriptive statistics as they remain *constant* throughout the entire observation period (standard deviation (`std`) = 0). Marrakesh’s backend had 16 missing values, probably due to a hardware issue or with the API. To correct this, we filled them with the last value observe.

Observing the statistical analysis of our data, we notice common findings :

- **Frequency and Anharmonicity** are always `null` and will be excluded from the analysis.
- All **duration** metrics are constant and never change over time. Since the duration represents something fixed by the constructor, this is not predictable.
- Since `rz_error` represents the virtual rotation, as defined earlier, it will always be 0 or null.
- Reset error was always be equal to 0, so unpredictable.
- **id_error**, **rx_error** and **sx_error** share identical values. They exhibits a correlation of 1. The randomized benchmarking sequence includes SX, ID, and X gates, and it is assumed their errors are the same. <https://quantum.cloud.ibm.com/docs/en/guides/qpu-information>
- **measure_error** and **readout_error** are identical metrics since `Measure_error` represents the error related to a qubit to be wrongly measured and the `Readout_error` represents the error of the **gates** responsible of this measure.

Correlation Among all the backend some correlation exhibits between features (*See in the appendix : Figure 7, Figure 8, Figure 9.*)

We noticed common finding, $T1$ and $T2$ are correlated in each backends (Fez: 0.75, Marrakesh: 0.70, Torino: 0.35). As expected, $T1$ is always greater than $T2$ through all backends and all time periods, following the theoretical constraint $T_2 \leq 2 \times T_1$ [6]. The *Readout error* and the *prob_meas0_prep1/prob_meas1_prep0* almost always exhibits a positive correlation above 0.5. This relation was expected since the readout error is often computed as the mean of `prob_meas1_prep0` and `prob_meas0_prep1`.

However, some relation appear in only specific backend. For example, in Marrakesh we have a negative correlation of -0.59 for the relation $T1$ and `id_error`, since this relation is lower than 0.1 in the other backends. Then, for a first analysis, we plot every feature independently to try to see some patterns. Linked with what we said before, we notice that the values of metrics don’t change always at the same frequency. As an example, the $T1$ and readout error change more often than the CZ and RZZ error.

Backend Performance Comparison The EDA allows us to notice that the three backends are not equal in terms of performance and behavior:

- **T1:** ibm_marrakesh achieves the highest relaxation time with a mean of $190 \mu s$, while ibm_fez reaches a maximum of only $149 \mu s$.
- **T2:** ibm_torino shows the best dephasing time with a mean of $135 \mu s$.
- **Readout error:** ibm_fez and ibm_marrakesh are comparable, while ibm_torino exhibits a readout error approximately three times higher on average.

Temporal Stability Comparing the distributions over two weeks and three months reveals consistent statistical properties, suggesting that the calibration behavior remain *stable over time*. T1 and T2 metrics appear relatively stable over time, while readout errors show significantly higher variability, particularly for ibm_marrakesh. Although gate durations are fixed by IBM and do not vary over time, they exhibit a notable correlation with T1 and T2 across backends. A longer readout duration is related to a higher T1 but a lower T2, suggesting a trade-off in IBM’s calibration strategy (Purcell effect [6]).

4 Modeling and Learning

In this section, we use the observations made during the exploratory data analysis (EDA) to prepare our data for modeling with a feature engineering part. Following this, we examine the predictability of the different calibration metrics and evaluate various models on our dataset collected over 126 days, at two levels of granularity: an hourly scale using the median of each feature across all qubits, and a daily scale computed as the median of the hourly dataset grouped by day.

4.1 Feature Engineering and Training Dataset

The EDA provided additional knowledge into the different features and their importance, which facilitated the modeling process. The previously presented analysis allowed us to reduce the feature space from 26 features to 9 relevant and consistent features for the learning part, in line with observations reported in other studies [2]. The set of features used for modeling is as follows : `calibration_time`, `t1`, `t2`, `readout_error`, `prob_meas0_prep1`, `prob_meas1_prep0`, `single_gate_error`, `cz_error` and `rzz_error`.

Following our EDA, a small numbers of features present outliers and those were concentrated in small areas. These anomalies values are probably due to an issue during the scraping from the API or are the result of technical errors in the physical device. However since these values are in the minority (below 1% of their features) and can affect the performance of model, we removed them. After visualize them using scatter plots during the EDA, we used the Inter-Quartile Range (IQR) method to numerically identify the outliers in each feature f , empirically adapting the quantile order (see Eq. 5).

$$IQR_f = q_{3,f} - q_{1,f}, \quad \forall f \in \mathcal{F} \quad (4)$$

$$\forall x \in f, \quad x = \begin{cases} \text{valid} & \text{if } x \in [q_{1,f} - 1.5 \times IQR_f, q_{3,f} + 1.5 \times IQR_f] \\ \text{outlier} & \text{otherwise} \end{cases} \quad (5)$$

Where $\mathcal{F}$ represents the set of features remaining from the feature engineering step, and $q_{1,f}$ and $q_{3,f}$ denote the first and third quartiles of feature $f \in \mathcal{F}$, empirically adjusted for each feature presenting visual outliers.

Guided by the literature on time-series forecasting and best practices for outlier handling [4, 11], we chose to replace the missing values induced by outlier removal using linear interpolation (see Eq. 6, where t_1 and t_2 are the two closest known timestamps to t , the value we want to fill in, and $f(t)$ the value of the feature $f \in \mathcal{F}$ at timestamp t).

$$f(t) = f(t_1) + \frac{f(t_2) - f(t_1)}{t_2 - t_1} \cdot (t - t_1), \quad (6)$$

4.2 Models

We considered three families of forecasters for our task:

- **Recurrent Neural Network:** Long Short-Term Memory (LSTM) & Gated Recurrent Unit (GRU)
- **Statistical:** SARIMAX
- **Hybrid (Neural and Statistical):** NeuralProphet

4.2.1 Recurrent Neural Networks (RNN)

Recurrent models are a family of neural architectures designed for sequential learning. By using hidden states, they can capture temporal dependencies, making them highly effective and well-suited for time series forecasting. More specifically for calibration data, where the current state of a qubit or a gate may be dependent on its past behavior. Two RNN architectures were considered. **Long Short-Term Memory** model are an improved version of the classic RNN that enables the learning of long-term and short-term dependencies. Employing a mechanism for managing long-term and short-term memory, this model introduces a concept of forgetting that controls the flow of information for long-term dependencies and addresses the issues of vanishing and exploding gradients [7]. Due to its memory capacity and its use of temporal dependencies, the LSTM model is widely used for time series forecasting. Furthermore, the use of this model in similar projects [8, 3] has encouraged its adoption for our task.

Much like the LSTM, the **Gated Recurrent Unit** model produces similar results while requiring less computation and fewer parameters by using hidden states instead of cells, hence this model has exhibited better results than LSTM for problems with small amount of data. Since we only have approximately 3,000 calibration records per backend for the hourly dataset and around 126 for the daily dataset, the GRU may be better suited to our data context, as it requires fewer parameters and converges faster [1].

4.2.2 Statistical Models

Given the performance of the neural network models, we shifted our focus to a non-neural, statistical model. This type of model is based on the decomposition of the time series using seasonality and stationarity of our data but requires less data, providing a statistical framework for capturing temporal dependencies. Note that all the following test were run on the daily dataset only.

Stationarity A stationary time series maintains statistical properties, such as the variance or the mean, constant over time. For non stationary time series, technique like differencing can be used to make it stationary by subtracting the previous observation from the current observation. One of the main requirements for using SARIMAX models is that our data be stationary. To verify this we performed an Augmented Dickey-Fuller test to check for the presence of a unit root [4, 10]. If the `p_value` calculated by this test is less than 0.05, the series is said to be stationary, which means that the variance and mean of our series are stable over time.

Backend	Feature	ADF p-value	Stationarity
ibm_fez	T1	0.0003	Stationary
	T2	0.0002	Stationary
	Readout error	0.0000	Stationary
	Prob meas0 prep1	0.0000	Stationary
	Prob meas1 prep0	0.0000	Stationary
	CZ error	0.0002	Stationary
	RZZ error	0.0000	Stationary
	Single gate error	0.0002	Stationary
ibm_marrakesh	T1	0.0141	Stationary
	T2	0.0191	Stationary
	Readout error	0.0035	Stationary
	Prob meas0 prep1	0.0029	Stationary
	Prob meas1 prep0	0.0001	Stationary
	CZ error	0.0552	<i>Non-stationary</i>
	RZZ error	0.0000	Stationary
	Single gate error	0.0902	<i>Non-stationary</i>
ibm_torino	T1	0.0134	Stationary
	T2	0.0003	Stationary
	Readout error	0.0000	Stationary
	Prob meas0 prep1	0.0000	Stationary
	Prob meas1 prep0	0.0000	Stationary
	CZ error	0.0000	Stationary
	RZZ error	0.0011	Stationary
	Single gate error	0.0004	Stationary

Table 4: ADF Stationarity Test Results by Backend and Feature

In our case, most of our metrics are considered stationary for our three backends with a fairly high level of confidence. Only the variables `cz_error` and `single_gate_error` on `ibm_marrakesh` exceed this threshold, suggesting non-stationary behavior that could require differentiation ($d=1$) (see Table 4).

Seasonality Seasonality refers to patterns that repeat over time at regular intervals (for daily data, these can be weekly, monthly, etc.). To detect seasonality in a time series, we analyzed the Autocorrelation Function (ACF) and the Partial-Autocorrelation Function (PACF), measuring the correlation between a current feature and its past values at different time lags [4]. Most of our features exhibit coefficients below 0.3 from the first lag, suggesting a very low correlation between the value of the current metrics with its past values [10]. The ACF and PACF analysis suggests that the signal does not exhibit highly exploitable memory or evident seasonal pattern, providing prior information about a seasonality term of $s = 0$ and a low autoregressive order p [4], which controls the number of past values used to forecast the current one in SARIMAX model.

Seasonal AutoRegressive Integrated Moving Average (SARIMAX) The *SARIMAX model* is a statistical framework designed for time series forecasting by extracting trends and seasonal variations from the data.

The *SARIMAX* model represents the combination of different statistical frameworks based on the following key principles:

- **Seasonality (S):** captures recurrent periodic patterns. Using methods like ACF, our series exhibits no evident seasonality. From this prior information, the hyperparameter s of our model will be set to 0.
- **Autoregression (AR):** decomposes the current value as a linear combination of p past values from our time series. Our prior analysis suggests a low value of p , since the series exhibits low memory.
- **Integration (I):** allows us to make our series stationary via a differentiation order d . The ADF test (see Table 4.2.2) suggests the series is stationary, hence the d term will be set to 0.
- **Moving Average (MA):** models the relationship between the current value and past prediction errors for the last q values. As noted in the literature [4], p fixes an upper bound for q . The usual practice to choose values of p and q is to search over a grid of values bounded by our prior analysis.
- **Exogenous Variables (X):** allows the use of external variables that provide additional information to improve predictions.

4.2.3 Neural Prophet

Given the limitations of *SARIMAX* model and *RNN*, we explored *NeuralProphet* an improved version of *Prophet* the statistical times series forecaster of Meta. It combine the structural decomposition of statistical models (such as *SARIMAX*) with the power of neural networks, enabling the capture of nonlinear dependencies and a statistical interpretation of our time series [12]. *Neural Prophet* relies on statistical components similar to *SARIMA* such as trend or seasonality but without exogenous variable, combined with neural autoregressive component allowing the model to capture non-linear pattern while keeping a statistical decomposition.

4.3 Experimental

In this part we will see how the experimentals part was runned, starting with a description of the software and hardware part of this projet for reproducibility. We then will talk about how the benchmark we runned and the parameters of each models. Finally, we will conclude with a presentation of the result and their analysis. All the element presented here, are present on Github <https://github.com/GwennGrs/Machine-Learning-for-Quantum-Machine-Calibration-Forecasting>.

4.3.1 Hardware and Benchmark Settings

All the code was written in Python 3.11, on a hardware under Ubuntu 24.04. The main Python package used for this project are `qiskit-ibm-runtime` 0.46, `pandas` 2.3.3, `numpy` 2.3.4, `scikit-learn` 1.8.0, `torch` 2.1.0, `neuralprophet` 0.9.0, `pmdarima` 2.1.1, and `statsmodel`. The implementation of neural models was carried out using PyTorch, based on concepts introduced in the Machine Learning 3 course (M. Bilasco, University of Lille). The architecture of the *SARIMAX* model was imported from the `statsmodel` package and the *NeuralProphet* model from the `neuralprophet` packages.

4.3.2 Training

To facilitate the training of the LSTM and GRU models, we have implemented an `Optimisation` class that automatize the training process, taking as parameters a model, a loss function (MSE), and an optimizer (Adam). For training our RNN models, our features were standardized to facilitate the interpretation of results and training, and split in an 80/20 ratio. For SARIMAX and NeuralProphet, the training process was managed by their respective Python libraries (`statsmodels` and `neuralprophet`).

To evaluate our models, we calculated various metrics during our benchmarking: the **MAE**, the standard metric most commonly used in the literature [8], the **RMSE**, which penalizes large errors and the R^2 , which indicates the proportion of variance explained. With $R^2 = 1$ if the model explains all the variance, 0 if it explains none, and less than 0 if it performs worse than average.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad (7)$$

with SS_{res} the sum of the error and SS_{tot} the total variance.

4.3.3 Benchmarking

Benchmarking was done on the complete calibration dataset that we extracted from October 29, 2025, to March 03, 2026, with a 1-hour frequency, for a total of 126 days of data (3,025 hours). Our modeling approach involves using the median of each feature across all qubits to start with lower complexity, revealing a smoother and more regular pattern than the 156 qubits per feature (reducing the number of features from 156×9 to 9), which was supposed to simplify the modelisation task and provides an initial approach to the problem.

We conducted benchmarks for two levels of granularity, starting with the dataset on hourly basis and then applying the same process on the daily basis dataset. For all the model the benchmark follow the same pattern, starting with a *Data Partitionning* chronologically into three subset with 80% of the data representing the training and validation set and 20% for the test set. Then for the *Model Training*, we train our model on each features from each backend using the empirically selected hyper parameters using only the past values of the features we want to predict as variables. Followed by the *Prediction* part using the previously trained model to predict value over the same time interval as the test set. Finally, the *Error calculation part* used the previous prediction to compute the evaluation metrics previously defined (defined in Section 4.3.2).

Model	Hyperparameter	Hourly	Daily
LSTM / GRU	Hidden dimension	10	20
	Hidden layers	1	1
	Learning rate	0.001	0.001
	Epochs	100	1000
SARIMAX	p (max)	3	5
	d	0	0
	q (max)	3	5
	s	0	0
NeuralProphet	n_lags	20	7
	n_forecasts		1
	Seasonality		False

Table 5: Hyperparameter settings per model and dataset granularity

The Table 5 shows the various hyperparameters used to train our models, ensuring reproducibility. For SARIMAX, p and q represent the upper bounds for the auto_arima search, $d = 0$ because our data is stationary, and $s = 0$ because there is no evident seasonality. To establish a baseline, we decided to train a simple linear regression. The parameter for the RNN models were chosen empirically, being the one providing the best performances. The choice of the hyperparameter for SARIMAX model during the benchmark was done using the auto_arima function which allows us to use the preliminary information obtained during the EDA to set a range of parameters and fix upper bound to test and identify the parameters that offer the best performance. Since the model is invariant to orders of magnitude, we have normalized the data to facilitate the interpretation of the results and speed up the search for the auto_arima function.

4.4 Result

In this section, we present an analysis of the benchmark results based on multiple evaluation metrics. We study the metrics computed for each feature and compare them to analyze the performance of each model across two different granularities.

Metric	GRU			LSTM			Linear			SARIMAX			NeuralProphet		
	MAE	RMSE	R^2	MAE	RMSE	R^2	MAE	RMSE	R^2	MAE	RMSE	R^2	MAE	RMSE	R^2
t1	0.061	0.214	0.933	0.051	0.213	0.933	0.054	0.225	0.929	0.815	0.963	-0.816	0.010	0.040	0.929
t2	0.055	0.192	0.932	0.047	0.185	0.935	0.055	0.192	0.931	0.852	1.075	-0.737	0.010	0.041	0.933
cz_error	0.081	0.195	0.901	0.082	0.212	0.871	0.054	0.179	0.927	1.025	1.153	-4.387	0.009	0.037	0.921
prob_meas0_prep1	1.012	1.218	-1.425	1.058	1.276	-1.677	0.265	0.494	0.684	3.515	4.510	-9.822	0.120	0.157	0.392
prob_meas1_prep0	0.981	1.087	-1.924	1.150	1.261	-3.054	0.337	0.445	0.534	0.588	0.746	-0.666	0.053	0.071	0.365
readout_error	1.115	1.279	-2.158	1.141	1.316	-2.323	0.304	0.480	0.687	2.198	2.882	-1.563	0.053	0.086	0.452
rzz_error	0.102	0.360	0.843	0.097	0.359	0.844	0.090	0.343	0.852	0.865	1.243	-4.724	0.011	0.060	0.858
single_gates	0.113	0.313	0.906	0.095	0.301	0.909	0.065	0.276	0.915	0.957	1.319	-0.311	0.013	0.057	0.913

Table 6: Comparison of models across calibration metrics provided by the benchmark, averaged over all backends (hours).

The metrics in Table 6 exhibit good performance, with $MAE < 0.1$ overall and variance ratios above 0.9 almost everywhere. However, as expected, Neural Prophet outperforms the other models on almost all metrics. The benchmark executed on NeuralProphet exhibits $MAE \leq 0.01$ for almost all features, while other models struggle to achieve MAE below 0.05. The more unstable behavior of the readout_error, prob_meas0_prep1 and prob_meas1_prep0 metrics makes patterns harder to extract, which is particularly noticeable with negative R^2 values for the RNN models and below 0.4 for NeuralProphet showing a lower proportion of variance explained. SARIMAX model exhibits the worst metrics, with $MAE \approx 1$ for most of the features as it consistently predicts the same values based on his precedent prediction.

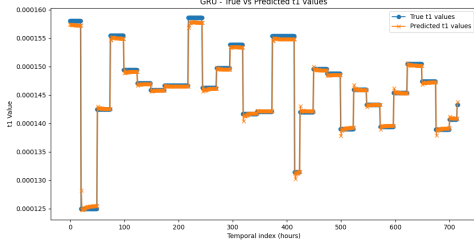


Figure 3: Forecasting of the T1 qubit median using a GRU model on hourly calibration data (ibm_fez, time lags: 2 steps)

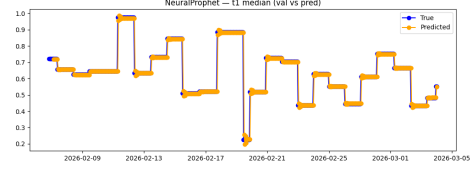


Figure 4: Forecasting of the T1 qubit median using Neural Prophet model on hourly calibration data (ibm_fez)

The prediction plot for the T1 calibration metrics on `ibm_fez` with GRU (see Figure 3) and NeuralProphet (see Figure 4) are representative of the behavior of all the metrics for the other models too. The predictions look visually satisfactory, however, the models only predicts the last value it saw. In this context, due to the way our data are distributed, the model predicts the correct value most of the time, making a mistake only when the value changes abruptly and no longer satisfies $\hat{t} = t - 1$. This fairly simple pattern to learn (called the naive approach) is the reason why all the models have equivalent performance. Since this holds true for nearly all of our features across all our backends (see in Figure 6), this explains the rather optimistic metrics in our benchmark, with the exception of `readout_error`, `prob_meas0_prep1`, and `prob_meas1_prep0`, which present more instable behavior. As explained in the EDA, these three features are very closely related and are the three features that change calibration most frequently. Therefore, the simplistic pattern seen in our other features is not present here, making prediction more complex.

Metric	GRU			LSTM			Linear			SARIMAX			NeuralProphet		
	MAE	RMSE	R^2	MAE	RMSE	R^2	MAE	RMSE	R^2	MAE	RMSE	R^2	MAE	RMSE	R^2
<code>cz_error</code>	0.798	0.924	-2.646	0.834	0.954	-3.258	0.856	0.964	-3.435	1.015	1.146	-3.425	0.193	0.225	-1.698
<code>prob_meas0_prep1</code>	1.866	2.016	-10.492	1.806	1.959	-9.697	0.792	1.014	-1.148	3.797	4.951	-1.872	0.837	0.956	-80.836
<code>prob_meas1_prep0</code>	2.139	2.262	-11.402	2.358	2.478	-14.086	1.273	1.425	-3.518	0.682	0.799	-0.326	0.131	0.168	-0.174
<code>readout_error</code>	1.643	1.764	-8.162	1.746	1.864	-9.465	0.789	0.948	-0.723	3.151	4.008	-8.341	0.205	0.316	-0.126
<code>rz_error</code>	0.725	1.060	-1.305	0.726	1.061	-1.300	0.744	1.099	-1.654	0.896	1.304	-4.318	0.277	0.343	-36.835
<code>single_gates</code>	0.860	1.101	-0.046	0.891	1.136	-0.093	0.867	1.182	-0.173	0.977	1.334	-0.287	0.282	0.339	-2.116
<code>t1</code>	0.675	0.809	-0.374	0.691	0.821	-0.412	0.716	0.854	-0.541	0.831	0.972	-0.902	0.143	0.177	-0.802
<code>t2</code>	0.650	0.850	-0.341	0.678	0.881	-0.429	0.667	0.865	-0.392	0.831	1.050	-0.705	0.210	0.246	-3.176

Table 7: Comparison of models across calibration metrics, averaged over all backends (daily dataset).

The Table 7 represent the evaluation metrics computed on our prediction with the different model on the daily basis dataset. For the RNN models, the best results obtained following an extensive hyperparameter tuning phase are less encouraging. While the LSTM model on hourly data showed an R^2 close to 0.9 for most features, indicating a high proportion of variance explained by the model in the case of the dataset based on daily medians, the R^2 obtained is very often close to 0 or well below 0 (R^2 for `prob_meas1_prep0` = -11.4), indicating a proportion of variance worse than predicting the mean. MAE performance has also deteriorated significantly, dropping from an MAE close to 0.06 for the three models on hourly $T1$ to an MAE greater than 0.6 for daily $T1$. While SARIMAX didn't provide better performance than the RNN models, Neural Prophet strongly outperform other models, exhibiting MAE and RMSE 3 or 4 times lower than the other models. However, the R^2 metrics for Neural Prophet were below 0 for all the features, showing almost

no proportion of variance explained.

To try to improve our model's performance, we attempted to forecast each feature using the others.

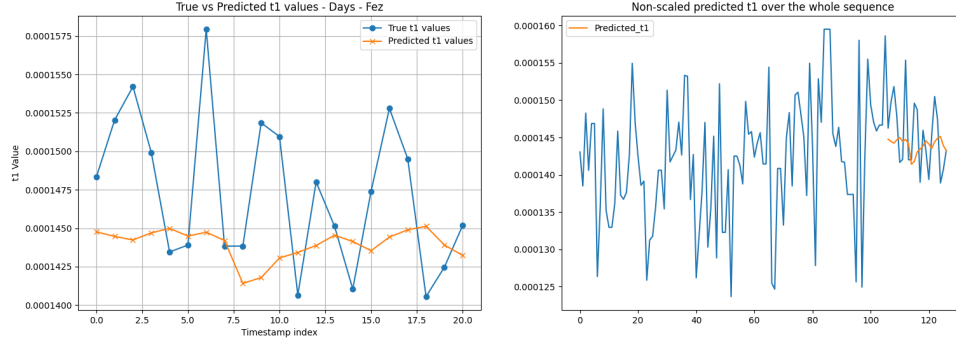


Figure 5: Forecasting of the T1 qubit median using a GRU model on daily calibration data (ibm_fez, time lags: 5 steps, using all the features)

The Figure 5 show the prediction of T1 for the backend fez with the model GRU adding the past values of the others features to predict our current value, this prediction is well representative of the behavior of other features for all the backends. The model's behavior shows improvement (see Figure 5). However, the model consistently predicts values close to the mean, indicating the difficulty for the model to predict current values even with the addition of other features past variables.

Additionally, we train the SARIMAX model using exogenous variables. The use of exogenous variables implies that we know the values of the other features at the moment of our prediction, which limits its use in our case.

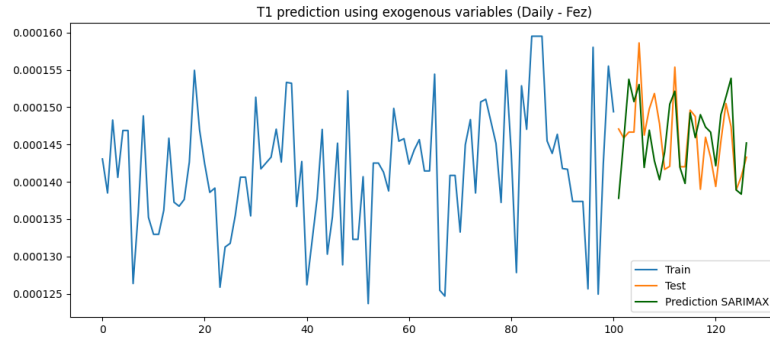


Figure 6: Prediction of T1 with SARIMAX using exogenous variables (Fez - Daily)

Figure 6 shows the SARIMAX model's predictions using exogenous variables; the T1 plot on the Fez machine clearly illustrates the predictions for features with strong correlation. This observation opens the

door to ideas for imputing missing values using the values of other features at that timestamp.

Metric	MAE_model	MAE_naive	MAE_mean	ratio_vs_naive	ratio_vs_mean
t1	0.143	0.104	0.109	1.491	1.297
t2	0.210	0.081	0.114	2.829	2.009
cz_error	0.193	0.125	0.100	1.490	1.724
prob_meas0_prep1	0.837	0.157	0.596	6.154	7.554
prob_meas1_prep0	0.131	0.110	0.119	1.281	1.236
readout_error	0.205	0.099	0.314	1.924	0.984
rzz_error	0.277	0.122	0.095	9.388	5.522
single_gates	0.282	0.192	0.203	1.747	1.696

Table 8: Comparison of calibration metrics for NeuralProphet, averaged over all the backends (days)

Since the NeuralProphet model outperforms most other models on the daily dataset, we decided to compare it only to a naive baseline and a mean prediction. Table 8 shows a comparison of the MSE between the different approaches. A ratio greater than 1 indicates that NeuralProphet underperforms the approach we compared it to. NeuralProphet outperforms the RNN and SARIMAX models on the daily dataset, achieving $MAE \approx 0.2$ for all the features (Table 8). However, the `ratio_vs_naive` column shows that for most features, NeuralProphet still performs worse than the naive model ($\hat{y}_t = y_{t-1}$), with ratios consistently above 1. This indicates that despite its more complex architecture, NeuralProphet struggles to outperform a simple baseline on the daily dataset, suggesting that the data does not contain easily exploitable patterns at this level of granularity.

5 Conclusion and Perspectives

The main objective of this project was to establish a complete pipeline, from the automated extraction of calibration data to its modeling. This project enabled the creation of a comprehensive dataset, divided into three parts: gates, qubits, and complete, providing a degree of modularity for our project and potential future applications. The code available on the GitHub repository <https://github.com/GwennGrs/Machine-Learning-for-Quantum-Machine-Calibration-Forecasting> will allow us to expand this dataset and continue this work.

In line with the state of the art, our work has highlighted the instability of current quantum machines, however the forecasting of calibration states appears possible. Among the various modeling approaches tested, NeuralProphet stood out by offering the most promising results, outperforming recurrent neural networks (RNNs) and classical statistical methods (SARIMAX). However, all models converge toward the naive pattern on the hourly dataset ($\hat{y}_t = y_{t-1}$), demonstrating once again the difficulty for models to move away from this approach and predicting when the data will change abruptly.

Despite modest results among all the models, the calibration metrics represented on the daily basis seems to exhibit temporal pattern. The results of Neural Prophet on the daily dataset encourage the possibility of forecasting calibration metrics over time. While, the GRU and LSTM models fail when switching to the daily dataset (3,025 data points per backend at 126) certainly due to a lack of data and/or to the complexity of our data. NeuralProphet is able to identify patterns and highlight meaningful relationships within the data, but it still struggles to outperform basic methods.

This project revealed several limitations and highlighted possible future directions. The repeated use of the median to aggregate our data may have introduced smoothing that masks patterns and causes a loss of information, a future direction for the project would be a qubit-wise approach, in order to study the behavior of the qubits independently of the rest of the machine. Finally, the findings of our project are in line with the state of the art [3, 10, 9], since forecasting effectively quantum calibrations over time appears possible, but in our view requires a more in-depth study of the behavior of qubits and calibrations over a longer time interval. The dataset will be continuously expanded, making it possible for anyone who wishes to continue the project in the future.

Personal Contributions Finally, this project allowed me to work during months on a fascinating field and to deepen my skills in time series analysis, data engineering, and modeling. This project was also a good opportunity to get initiated and more familiar about quantum computing, a challenging but fascinating subject. I learned how to build a project step by step, plan its progress over time, and share my results with others. This project has allowed me to acquire essential skills in both data science practice and research, teaching me its demands and how to deal with uncertainty regarding experimental results.

Acknowledgments I would like to express special thanks to Dr. DAHI for his attentive guidance, availability, rigor, and support throughout the development of this project.

References

- [1] Oluwatomisin Arokodare. “Leveraging Machine Learning for Enhanced Predictive Accuracy in Time Series Forecasting: A Comparative Analysis of LSTM and GRU Models”. Mar. 18, 2024. DOI: 10.13140/RG.2.2.30014.80965.
- [2] Zakaria Abdelmoiz Dahi, Francisco Chicano, and Gabriel Luque. “An Evolutionary Deep Learning Approach for Efficient Quantum Algorithms Transpilation”. In: *Proceedings, Part II of the 27th European Conference, EvoApplications 2024, Held as Part of EvoStar 2024, Aberystwyth, UK, April 3–5, 2024*. Vol. 14635. Lecture Notes in Computer Science. Aberystwyth, United Kingdom: Springer Nature Switzerland, Apr. 2024, pp. 240–255. DOI: 10.1007/978-3-031-56855-8_15. URL: <https://hal.science/hal-04900875> (visited on 11/21/2025).
- [3] Zakaria Abdelmoiz Dahi et al. “Optimising Quantum Calculations Reliability via Machine Learning: The IBM Case Study”. In: *2025 IEEE International Conference on Quantum Artificial Intelligence (QAI)*. 2025 IEEE International Conference on Quantum Artificial Intelligence (QAI). Nov. 2025, pp. 308–315. DOI: 10.1109/QAI63978.2025.00055. URL: <https://ieeexplore.ieee.org/document/11344609> (visited on 03/24/2026).
- [4] Fatoumata Dama and Christine Sinoquet. “Time Series Analysis and Modeling to Forecast: a Survey”. In: arXiv:2104.00164 (Sept. 26, 2021). DOI: 10.48550/arXiv.2104.00164. arXiv: 2104.00164[cs]. URL: <http://arxiv.org/abs/2104.00164> (visited on 02/03/2026).
- [5] Yuta Hirasaki et al. “Detection of temporal fluctuation in superconducting qubits for quantum error mitigation”. In: *Applied Physics Letters* 123.18 (Oct. 30, 2023), p. 184002. ISSN: 0003-6951, 1077-3118. DOI: 10.1063/5.0166739. arXiv: 2307.04337[quant-ph]. URL: <http://arxiv.org/abs/2307.04337> (visited on 03/28/2026).
- [6] Philip Krantz et al. “A Quantum Engineer’s Guide to Superconducting Qubits”. In: *Applied Physics Reviews* 6.2 (June 1, 2019), p. 021318. ISSN: 1931-9401. DOI: 10.1063/1.5089550. arXiv: 1904.06560[quant-ph]. URL: <http://arxiv.org/abs/1904.06560> (visited on 02/03/2026).
- [7] “Long Short-Term Memory”. In: *ResearchGate* (). DOI: 10.1162/neco.1997.9.8.1735. URL: https://www.researchgate.net/publication/13853244_Long_Short-Term_Memory (visited on 03/24/2026).
- [8] Laura Rodríguez-Soriano, Francisco Garcia-Herrero, and Carmen G. Almudéver. “Predicting the Temporal Variability of Error Rates in Superconducting Quantum Processors: Mitigating Error Accuracy Loss in Queue-Based Quantum Computing Platforms [Focus: Quantum Software and its Engineering]”. In: *IEEE Software* 42.5 (Sept. 2025), pp. 90–97. ISSN: 1937-4194. DOI: 10.1109/MS.2025.3572982. URL: <https://ieeexplore.ieee.org/document/11015470> (visited on 02/03/2026).
- [9] Laura Rodríguez-Soriano, Francisco Garcia-Herrero, and Carmen G. Almudéver. “Predicting the Temporal Variability of Error Rates in Superconducting Quantum Processors: Mitigating Error Accuracy Loss in Queue-Based Quantum Computing Platforms [Focus: Quantum Software and its Engineering]”. In: *IEEE Software* 42.5 (Sept. 2025), pp. 90–97. ISSN: 1937-4194. DOI: 10.1109/MS.2025.3572982. URL: <https://ieeexplore.ieee.org/document/11015470> (visited on 03/24/2026).

- [10] Mohammadreza Saghaei and Lamine Mili. “Predictive Time-Series Analysis of Single-Qubit Quantum Circuit Outcomes for a Superconducting Quantum Computer: Forecasting Error Patterns”. In: *IEEE Access* 13 (2025), pp. 40115–40132. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3546259. URL: <https://ieeexplore.ieee.org/document/10906565> (visited on 02/03/2026).
- [11] *Time Series Analysis: Forecasting and Control, 5th Edition*, by George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel and Greta M. Ljung, 2015. Published by John Wiley and Sons Inc., Hoboken, New Jersey, pp. 712. ISBN: 978-1-118-67502-1. URL: https://www.researchgate.net/publication/299459188_Time_Series_Analysis_Forecasting_and_Control5th_Edition_by_George_E_P_Box_Gwilym_M_Jenkins_Gregory_C_Reinsel_and_Greta_M_Ljung_2015_Published_by_John_Wiley_and_Sons_Inc_Hoboken_New_Jersey_pp_712_ISBN_ (visited on 03/24/2026).
- [12] Oskar Triebe et al. *NeuralProphet: Explainable Forecasting at Scale*. Nov. 29, 2021. DOI: 10.48550/arXiv.2111.15397. arXiv: 2111.15397[cs]. URL: <http://arxiv.org/abs/2111.15397> (visited on 03/25/2026).

6 Appendix

The appendix allows for the inclusion of visual evidence in the report's analyses or the sharing of large tables. The various materials presented in the appendix will primarily serve as supporting documentation for the EDA.

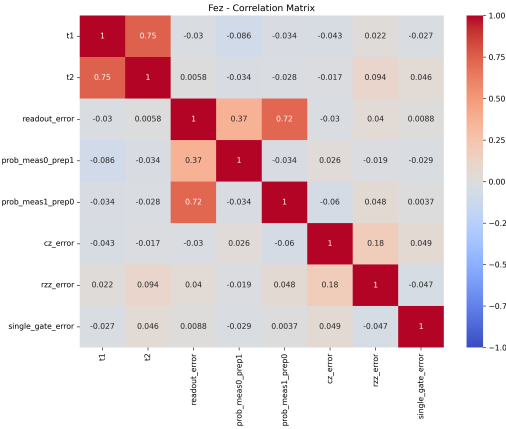


Figure 7: Correlation matrix – Fez on data scraped for 42 days, after feature engineering.

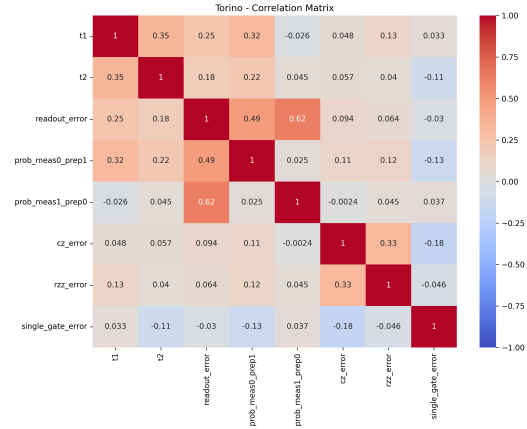


Figure 8: Correlation matrix – Torino on data scraped for 42 days, after feature engineering.

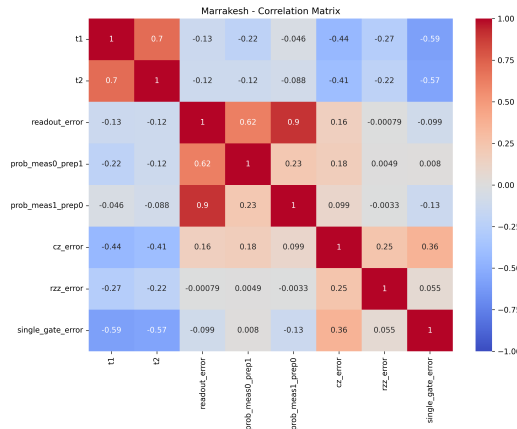


Figure 9: Correlation matrix – Marrakesh on data scraped for 42 days, after feature engineering.

Figure 7, Figure 8, and Figure 9 show the correlation matrix, calculated numerically, between the features of each backend following the feature engineering stage. This illustrates both similar behaviors across the backends and correlations specific to each backend. The redder the value, the stronger the positive correlation; a blue value represents a negative correlation.

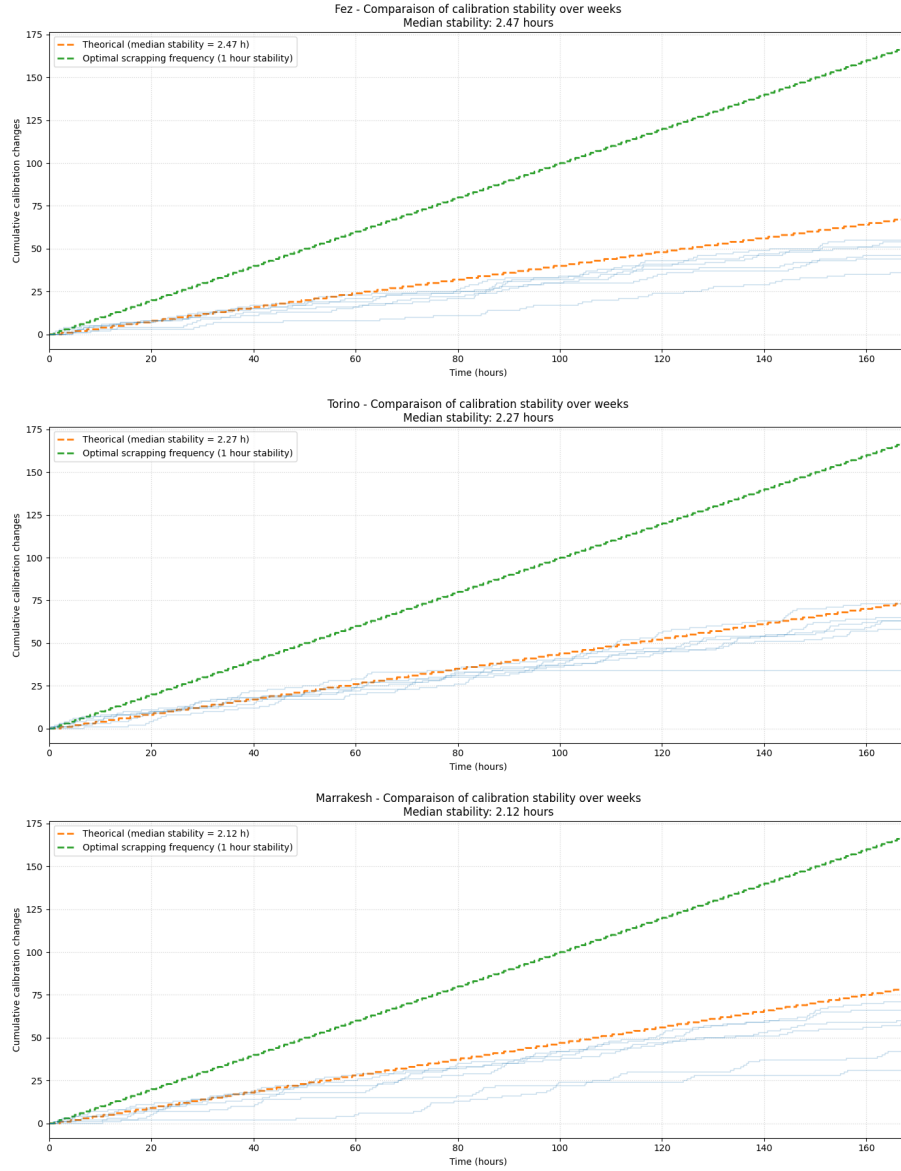


Figure 10: Visual analysis of the calibration stability of the three backends with a week-by-week comparison of cumulative calibration changes with the median of the stability time for each of them and our theoretical optimal scrapping frequency of one hour, 42-day dataset scraped with a frequency of 1 min.

Figure 10 served as the primary tool for providing visual evidence of the need to reduce the frequency of scraping. The three plots show similar patterns regarding the cumulative change over a one-week period. This plot shows the breakdown of each backend over the 42-day period, broken down by week.

	t1	t2	readout_error	frequency	anharmonicity	prob_meas0_prep1	prob_meas1_prep0	readout_length
count	12098.00	12098.00	12098.00	0.00	0.00	12098.00	12098.00	12098.00
mean	0.000141	0.000100	0.009489	NaN	NaN	0.013338	0.005665	0.000002
std	0.000008	0.000006	0.000456	NaN	NaN	0.000454	0.000731	0.000000
min	0.000124	0.000087	0.008057	NaN	NaN	0.012329	0.003418	0.000002
25%	0.000135	0.000098	0.009277	NaN	NaN	0.013062	0.005493	0.000002
50%	0.000142	0.000100	0.009460	NaN	NaN	0.013428	0.005859	0.000002
75%	0.000146	0.000103	0.009766	NaN	NaN	0.013672	0.006104	0.000002
max	0.000154	0.000116	0.010559	NaN	NaN	0.015137	0.006958	0.000002

Table 9: Descriptive statistics of qubit metrics for ibm_fez

	id_error	rx_error	rz_error	sx_error	cz_error	rzz_error	reset_error	measure_error
count	12098.00	12098.00	12098.00	12098.00	12098.00	12098.00	0.00	12098.00
mean	0.0284	0.0284	0.0000	0.0284	0.2684	0.2632	NaN	0.9489
std	0.0014	0.0014	0.0000	0.0014	0.0078	0.0251	NaN	0.0456
min	0.0264	0.0264	0.0000	0.0264	0.2575	0.2446	NaN	0.8057
25%	0.0273	0.0273	0.0000	0.0273	0.2638	0.2513	NaN	0.9277
50%	0.0285	0.0285	0.0000	0.0285	0.2666	0.2560	NaN	0.9460
75%	0.0296	0.0296	0.0000	0.0296	0.2725	0.2614	NaN	0.9766
max	0.0320	0.0320	0.0000	0.0320	0.2890	0.3593	NaN	1.0559

Table 10: Descriptive statistics of gate metrics for ibm_fez (*error values expressed in %*)

	t1	t2	readout_error	frequency	anharmonicity	prob_meas0_prep1	prob_meas1_prep0	readout_length
count	12098.00	12098.00	12098.00	0.00	0.00	12098.00	12098.00	12098.00
mean	0.000182	0.000135	0.030076	NaN	NaN	0.026215	0.029926	0.000002
std	0.000007	0.000004	0.001914	NaN	NaN	0.001532	0.003287	0.000000
min	0.000162	0.000123	0.022705	NaN	NaN	0.022705	0.017578	0.000002
25%	0.000177	0.000132	0.029053	NaN	NaN	0.025146	0.029297	0.000002
50%	0.000182	0.000135	0.030151	NaN	NaN	0.026123	0.030273	0.000002
75%	0.000186	0.000137	0.031372	NaN	NaN	0.027344	0.031738	0.000002
max	0.000192	0.000141	0.034668	NaN	NaN	0.030273	0.037354	0.000002

Table 11: Descriptive statistics of qubit metrics for ibm_torino

	id_error	rx_error	rz_error	sx_error	cz_error	rzz_error	reset_error	measure_error
count	12098.00	12098.00	12098.00	12098.00	12098.00	12098.00	0.00	12098.00
mean	0.0312	0.0312	0.0000	0.0312	0.2657	0.2673	NaN	3.0076
std	0.0013	0.0013	0.0000	0.0013	0.0074	0.0065	NaN	0.1914
min	0.0288	0.0288	0.0000	0.0288	0.2515	0.2557	NaN	2.2705
25%	0.0302	0.0302	0.0000	0.0302	0.2610	0.2623	NaN	2.9053
50%	0.0309	0.0309	0.0000	0.0309	0.2647	0.2664	NaN	3.0151
75%	0.0323	0.0323	0.0000	0.0323	0.2707	0.2699	NaN	3.1372
max	0.0343	0.0343	0.0000	0.0343	0.3038	0.2839	NaN	3.4668

Table 12: Descriptive statistics of gate metrics for ibm_torino (*error values expressed in %*)

	t1	t2	readout_error	frequency	anharmonicity	prob_meas0_prep1	prob_meas1_prep0	readout_length
count	12098.00	12098.00	12098.00	0.00	0.00	12098.00	12098.00	12098.00
mean	0.000182	0.000102	0.026192	NaN	NaN	0.028542	0.020427	0.000003
std	0.000011	0.000009	0.071256	NaN	NaN	0.057240	0.108830	0.000000
min	0.000151	0.000080	0.009277	NaN	NaN	0.014893	0.001953	0.000003
25%	0.000180	0.000097	0.010803	NaN	NaN	0.016968	0.003418	0.000003
50%	0.000185	0.000103	0.011353	NaN	NaN	0.017578	0.003662	0.000003
75%	0.000191	0.000110	0.011963	NaN	NaN	0.018799	0.003906	0.000003
max	0.000197	0.000116	0.450439	NaN	NaN	0.421997	0.783203	0.000003

Table 13: Descriptive statistics of qubit metrics for ibm_marrakesh

	id_error	rx_error	rz_error	sx_error	cz_error	rzz_error	reset_error	measure_error
count	12098.00	12098.00	12098.00	12098.00	12098.00	12098.00	0.00	12098.00
mean	0.0326	0.0326	0.0000	0.0326	0.2618	0.2655	NaN	2.6192
std	0.0025	0.0025	0.0000	0.0025	0.0135	0.0367	NaN	7.1256
min	0.0272	0.0272	0.0000	0.0272	0.2377	0.2312	NaN	0.9277
25%	0.0310	0.0310	0.0000	0.0310	0.2509	0.2456	NaN	1.0803
50%	0.0324	0.0324	0.0000	0.0324	0.2637	0.2605	NaN	1.1353
75%	0.0335	0.0335	0.0000	0.0335	0.2702	0.2669	NaN	1.1963
max	0.0396	0.0396	0.0000	0.0396	0.3028	0.4354	NaN	45.0439

Table 14: Descriptive statistics of gate metrics for ibm_marrakesh (*error values expressed in %*)

Table 9, Table 10, Table 11, Table 12, Table 13 and Table 14 provide a statistical description of each value for each backend based on data extracted at 1-minute intervals over 42 days. This allows us to highlight values that are stagnant or consistently zero, or to compare features across backends.